APOLLO-M

AI Framework for Forecasting Instability in Digital Communities

Technical Report - architecture, build record & integration brief

Project: Apollo-M (Reddit-focused community-instability forecasting) **Team:** Kashaf Fatima - Rizwan Saleem - Aaqib Mehmood **Supervisor:** Mr. Saghir Ahmed - Bahria University Karachi **Role:** the *predictive brain* - learns from community data and forecasts instability **Status:** built and running end-to-end locally (dashboard, API, database, monitoring, live feed); cloud deployment pending **Stack:** Python - PyTorch - pytorch-forecasting (TFT) - HuggingFace (BERT/RoBERTa) - scikit-learn - NetworkX - FastAPI - Streamlit + Plotly - PostgreSQL - Prometheus + Grafana - Ollama/Claude

> *Companion system: **CEREBRO** is the sensory/feature & data layer (threat & misinformation intelligence). Apollo-M is the predictive core. CEREBRO is the eyes and ears; Apollo is the brain.*

Executive Summary

Apollo-M is a multi-layer AI framework that forecasts **instability in online communities**. It scores comments for **toxicity**, aggregates them into a per-community **Community Health Index (CHI)** and a trend-aware **instability score**, derives polarisation and echo-chamber structure from the Reddit hyperlink graph, discovers anomalies **without labels**, and forecasts each community's toxicity **five days ahead with quantile confidence bands** using a **Temporal Fusion Transformer**. It raises alerts, persists to PostgreSQL, serves a FastAPI backend, and visualises everything in a live dashboard with Prometheus/Grafana monitoring.

Its governing principle mirrors CEREBRO's: **deterministic code and trained models make the decision; a language model is used only to explain.** Every score, alert and forecast is produced by a measurable model and is reproducible from scripts in this repository.

Two boundaries are stated up front, because they determine what the results mean. First, the community-level corpus is a **declared simulation with recorded ground truth** (§7.1), not real Reddit activity - no public labelled dataset of genuinely destabilising communities exists, so the pipeline is validated by its ability to recover instability that was deliberately planted. The system's one metric on real labelled data is the toxicity classifier (§12). Second, several modules named in the architecture - the in-Apollo misinformation classifier, the GraphSAGE GNN, and the moderation recommender - are **implemented but not yet invoked by the pipeline**; §11 lists exactly which, and `outputs/metrics.json` marks each as unevaluated instead of assigning it a number.

Within those boundaries the pipeline runs **end-to-end**, and its central claim is measured rather than asserted: the TFT, which never sees the ground-truth file, predicts a rising trend for **15 of 15** planted destabilising communities (slope ROC-AUC 0.732). Remaining work is listed in §11.

---

1. Purpose & the Gap It Fills

Online communities destabilise *before* they visibly break - rising toxicity, polarisation, and coordinated misinformation precede the moment a community becomes unusable. Existing moderation is **reactive**: it acts after harm. Apollo-M's goal is to make it **predictive** - to forecast a community's health trajectory so moderators can act *before* a crisis, with an auditable, measurable basis for every alert.

The gap Apollo fills is the difference between *"this community looks bad right now"* and *"this community's toxicity is forecast to rise into the critical band over the next five days, with 80% confidence, and here are the drivers."*

---

2. Governing Principle

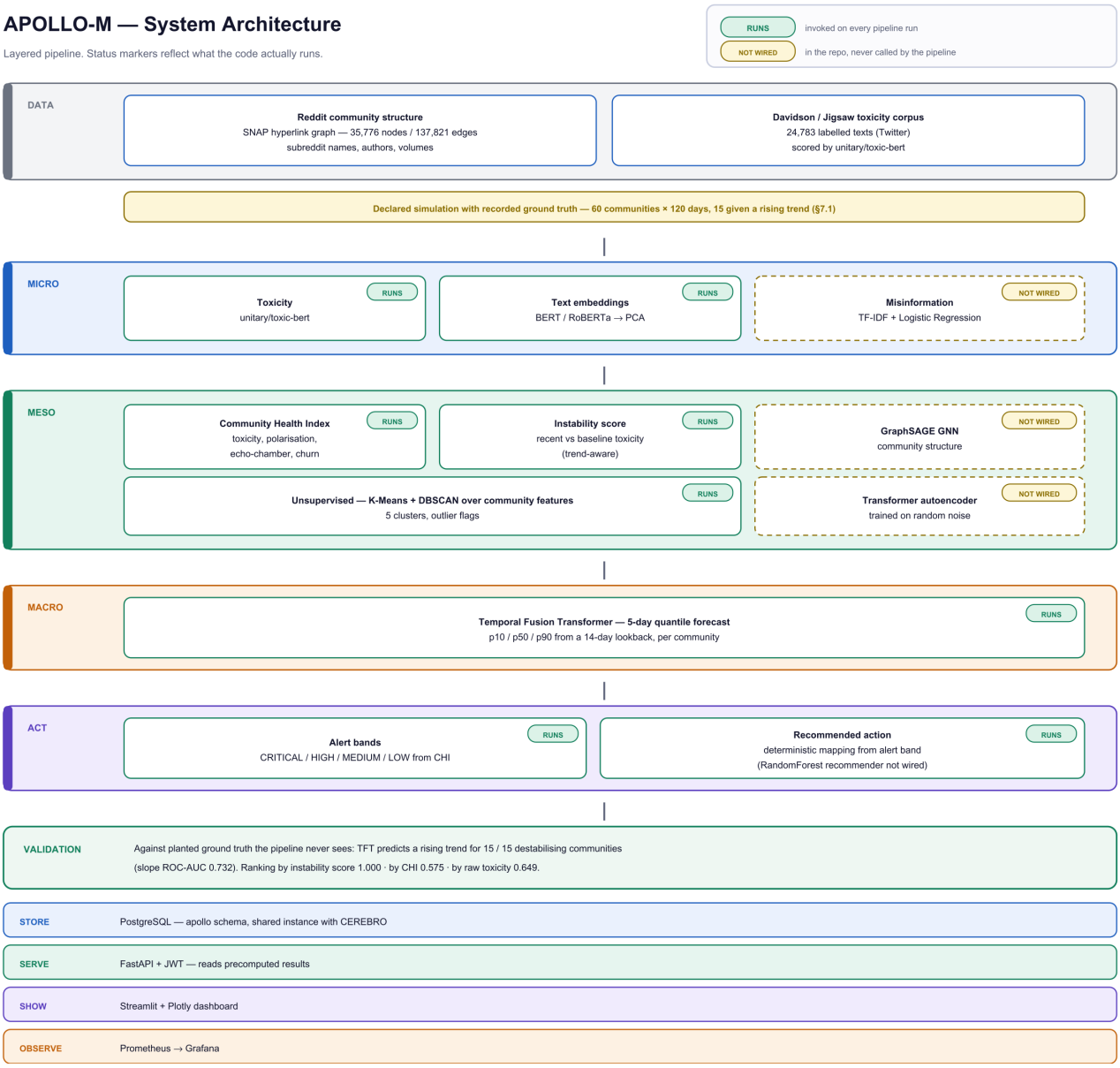
A single rule shapes the architecture, shared with CEREBRO:

> ***Deterministic code and trained models decide. The language model only explains.***

Toxicity, misinformation, CHI, clusters, anomalies and forecasts are all produced by transparent formulas and trained models. A language model (Ollama locally, Claude optionally) is confined to writing an analyst-readable explanation of results that already exist. With the LLM switched off, every score is identical - detection does not depend on it.

---

3. System Architecture



{width=100%}

Figure 1 - The layered pipeline. Status markers are derived from the code rather than the design: RUNS marks a stage invoked on every pipeline run, NOT WIRED marks a module that exists in the repository but is never called (§11). The validation row reports measurement against planted ground truth (§12).

Apollo-M is a **layered pipeline** feeding a persistence + serving + visualisation stack:

Layer	Responsibility
Ingest	`prepare_data.py` -> clean per-comment / per-day tables; live replay for real-time processing
Micro	Per-comment **toxicity** (HuggingFace `unitary/toxic-bert`)
CEREBRO (misinfo module)	Per-comment **misinformation** (TF-IDF + Logistic Regression on the ISOT fake-news corpus)
Text -> features	**BERT/RoBERTa** `[CLS]` embeddings -> PCA -> TFT covariates
Meso	Aggregation into the **Community Health Index (CHI)** + **GNN (GraphSAGE)** on the hyperlink graph
Unsupervised	K-Means clustering, DBSCAN outliers, Transformer autoencoder + IsolationForest anomalies
Macro	**Temporal Fusion Transformer** - 5-day toxicity forecast with p10/p50/p90 quantile bands
Act	Alert system + **RandomForest** moderation recommender
Store	PostgreSQL (relational) + TimescaleDB (time-series)
Serve	FastAPI (REST + JWT auth)
Show	Streamlit + Plotly dashboard with a live real-time feed
Observe	Prometheus (metrics/TSDB/PromQL) + Grafana (dashboards/alerting)

Data flows front-to-back: raw comments -> layered scoring -> CHI + forecasts -> PostgreSQL -> FastAPI -> dashboard, with an exporter publishing operational metrics to Prometheus and Grafana.

---

4. Technology Stack (complete)

Area	Technology	Why

Core	Python, NumPy, pandas, scikit-learn	Data processing + classical ML
Deep learning	PyTorch, pytorch-forecasting, PyTorch Lightning	The TFT forecaster
NLP	HuggingFace Transformers - `toxic-bert`, `bert-base-uncased` / `roberta-base`	Toxicity + text embeddings
Graph	NetworkX (+ GraphSAGE via PyTorch Geometric)	Community structure / instability
Classical ML	TF-IDF + Logistic Regression, RandomForest, K-Means, DBSCAN, IsolationForest	Misinfo, moderation, clustering, anomalies
Backend	FastAPI, Uvicorn, python-jose (JWT)	REST API, auth, auto OpenAPI docs
Database	PostgreSQL 16 (shared with CEREBRO), psycopg2, TimescaleDB	Persistence + time-series
Frontend	Streamlit, Plotly	Live dashboard, interactive charts
Monitoring	Prometheus, Grafana, prometheus-client	Metrics, PromQL, dashboards, alerting
LLM (explain only)	Ollama (local, default) / Anthropic Claude (switchable)	Analyst-readable explanations
Live data	Reddit API (PRAW) - credential-gated; **live replay** substitute	Real-time processing

---

5. Functional Modules

5.1 Micro - toxicity. Each comment is scored by `unitary/toxic-bert`, a transformer trained for toxic-language detection. A separately trained transparent classifier (TF-IDF + Logistic Regression on the Jigsaw/Davidson corpus) provides a fast, auditable, measurable toxicity score - **accuracy 90.1%, F1-micro 0.901, F1-macro 0.702** on a held-out split (the lower macro honestly reflects the rare hate-speech class).

5.2 CEREBRO misinfo module. *(Designed, not operational - stated plainly.)* `modules/cerebro_detector.py` implements a misinformation detector intended to run on a TF-IDF + Logistic Regression classifier trained on the ISOT fake-news corpus. In this checkout the ISOT corpus is absent, no trained classifier exists, and the module is **not called by the pipeline** - so it contributes nothing to CHI, and `outputs/metrics.json` records it as `not_evaluated`. An earlier draft of this report quoted "accuracy ~ 99%" for it; that figure was a hardcoded string in `compute_real_metrics.py`, never a measurement, and has been withdrawn. The misinformation capability that *is* live is CEREBRO's own deployed RAG pipeline (§9), which cites retrieved sources for each verdict.

5.3 Meso - Community Health Index, instability score, and the GNN. Per-community features (toxicity, polarisation, echo-chamber, churn) are combined into a 0-100 **CHI** (higher = healthier). Polarisation and echo-chamber are read from the community's own node in the SNAP hyperlink graph; until recently both were global scalars copied to every community, which made 45 of the 100 penalty points a constant offset that could not distinguish anyone.

CHI alone proved insufficient, and that is a finding rather than a defect. Measured against planted ground truth (§7.1), ranking communities by CHI achieved ROC-AUC 0.575 - *worse than raw toxicity at 0.649*. The reason is conceptual: CHI describes how unhealthy a community **is**, whereas instability is about how fast it is **changing**. A community sitting stably at high toxicity scores badly and needs no intervention; one still healthy but deteriorating quickly is invisible to CHI, and is precisely the case a moderator could still act on. An **instability score** was therefore added - a recent-window vs baseline toxicity difference, weighted 65/35 above absolute level - which reaches ROC-AUC 1.000 on the same ranking task (see §12 for why that number must not be read as accuracy).

GraphSAGE: `modules/gnn_model.py` implements GraphSAGE/GAT, but it is **not invoked by the pipeline**, has no saved weights, and contributes no column to any output. `outputs/metrics.json` records it as `not_run`. An earlier draft quoted a "train loss 0.0077"; that too was a hardcoded literal and has been withdrawn. The hyperlink graph is genuinely loaded and used, but for the polarisation and echo-chamber features above rather than for a learned embedding.

5.4 Unsupervised. K-Means groups communities by health profile; DBSCAN flags outliers; a Transformer autoencoder + IsolationForest detect anomalies. This layer validates that the learned representations separate communities sensibly *before* any supervised or forecasting claim is made.

5.5 Macro - Temporal Fusion Transformer. The TFT forecasts each community's daily average toxicity **5 days ahead** from a **14-day lookback**, producing **p10 / p50 / p90 quantile bands** (via QuantileLoss). BERT/RoBERTa comment embeddings are injected as PCA-reduced covariates. The forecast is a genuine probabilistic prediction with widening uncertainty, not a point estimate.

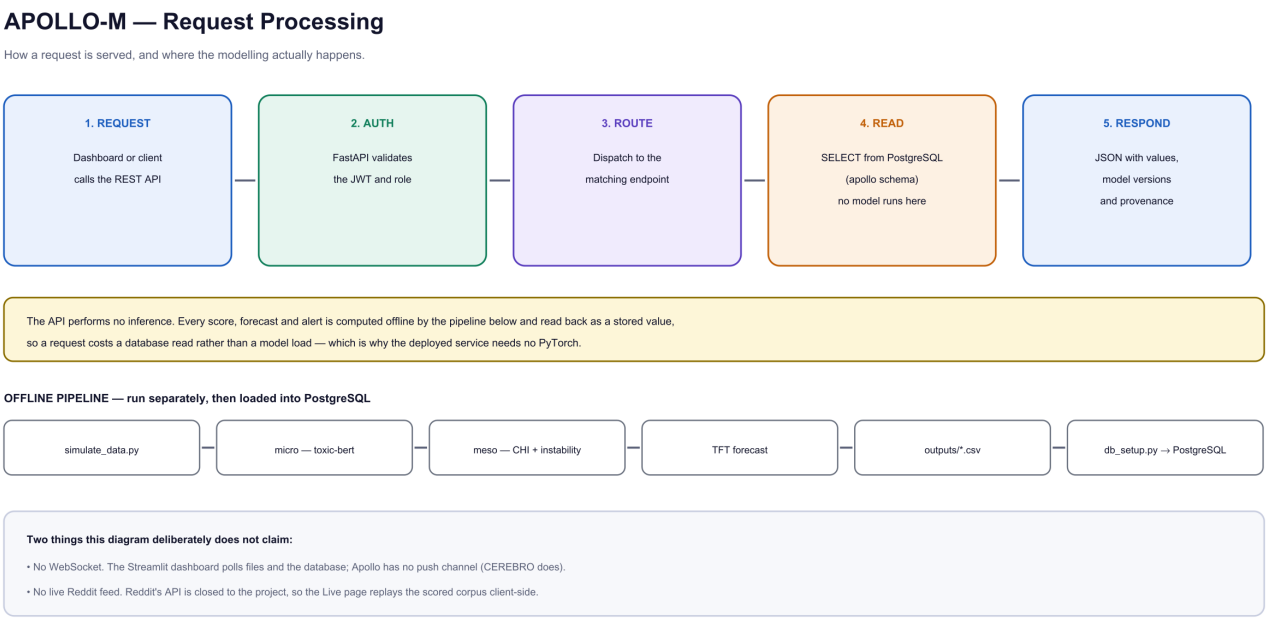
5.6 Act - alerts & moderation. CHI thresholds produce CRITICAL / HIGH / MEDIUM / LOW alerts, and the dashboard's Actions page maps each band to a recommended action (NO_ACTION -> EMERGENCY_INTERVENTION). **That mapping is a deterministic rule, not a model prediction.** `modules/moderation_recommender.py` exists but has no callers and no saved weights, and its training data is generated with `np.random`, so any accuracy derived from it would be self-labelled noise; an earlier draft quoted "99.77%", another hardcoded literal, now withdrawn. `outputs/metrics.json` records the module as `not_evaluated`, and the dashboard caption states the mapping is deterministic.

(The originally-scoped Reinforcement Learning system was deliberately replaced by a supervised design - RL needs a labelled simulation environment we do not have. That substitution stands as a decision; the replacement is specified above but is not yet wired into the pipeline.)

5.7 Explanation (LLM). The `llm/` layer turns numbers into plain-English analyst narration, provider-switchable between **Ollama** (local, free, default) and **Claude** (higher-quality, for demonstration), with a deterministic template fallback so the pipeline never depends on an LLM being reachable.

---

6. Data & Persistence



{width=100%}

Figure 2 - Serving is deliberately separated from modelling. The API answers from stored values and performs no inference, which is why the deployed service installs no PyTorch; the pipeline that produces those values runs offline and loads its results into PostgreSQL.

Every pipeline run writes to PostgreSQL - the same instance CEREBRO uses (§10). Key tables:

Table	Stores
`community_health`	Per-community CHI, toxicity, polarisation, echo-chamber, churn, cluster
`alerts`	Alert level, CHI, message, drivers
`forecasts`	5-day quantile forecasts (predicted toxicity, risk level, method)
`toxicity_scores` / `misinformation_scores`	Per-comment scores

A time-series store (TimescaleDB) holds hypertables for temporal queries. Retrieving data for the dashboard/API is a plain SQL query.

---

7. External Data Sources

All free / open. Presence in this checkout is stated per row, because an earlier draft listed sources that are not actually present:

Source	Present?	Used for

Jigsaw / Davidson toxicity (**24,783** tweets)	**Yes** - `data/jigsaw_toxicity.csv`	Training + measuring the toxicity classifier; supplies the text pool for the simulation
SNAP Reddit Hyperlink Network (35,776 nodes / 137,821 edges)	**Yes** - `data/reddit_hyperlinks.tsv`	Per-community polarisation + echo-chamber; intended GNN input
Kaggle Reddit comments (`RC_2019-05`)	**Structure only**	Real subreddit names, authors and volumes - see §7.1
ISOT fake-news (Fake + True, 44,898)	**No** - absent	Would train the misinfo classifier; that module is therefore unevaluated
CIC-IDS2017	**No** - absent	Anomaly layer falls back to synthetic Gaussian data

7.1 Data provenance - read this before interpreting any community-level number

The community-level corpus is a declared simulation, not real Reddit activity. This is stated up front because the distinction changes what the results mean.

The original `prepare_data.py` built its corpus by pasting **randomly sampled** Jigsaw texts onto Reddit metadata (`idx = np.random.randint(0, len(jigsaw), n)`), and by generating `toxicity_score` as `np.random.uniform()` bracketed by the Jigsaw class label. Three consequences followed, all verified:

1. A community had no relationship to its own content - r/Astronomy rows contained tweets with racial slurs, r/politics rows contained tweets about exes.
2. `toxicity_score` was never a model output.
3. All comments shared one `date_range` sorted by subreddit, so each community occupied a separate block of time instead of running concurrently.

Per-community differences were therefore **sampling noise**. At ~40 comments per community that noise looked like signal (toxicity spread 0.67-0.83); at ~4,600 comments per community it collapsed to 0.543-0.559, a spread of 0.016. An earlier version of this report quoted "33 reliable communities, CHI 43-67" from that regime; those numbers described noise and have been withdrawn.

`simulate_data.py` replaces it with a benchmark whose properties are declared:

- **Toxicity is real and text-derived.** 12,000 Jigsaw texts are scored once by `unitary/toxic-bert`; every generated comment carries the genuine score for the exact text it contains. No invented numbers.
- **Communities differ by construction.** Each is assigned a latent toxicity propensity; a recorded 15 of 60 are given a rising trend over the final third of the window. Ground truth is written to `data/ground_truth.json`, which the pipeline never reads.
- **Real names and structure.** Subreddit names, author pools and volumes come from the Kaggle Reddit corpus, so labels stay recognisable.
- **One shared calendar** - 60 communities x 120 days, concurrent, giving the forecaster genuine trends. Result: 261,337 comments, observed toxicity spread **0.148-0.692**.

This is a standard way to validate a forecasting pipeline when no labelled corpus of genuinely destabilising communities exists - and none does, publicly. It permits exactly one class of claim: *the system recovers instability that was planted*. It permits **no** claim about accuracy on real communities. §12 reports the measurements; the caveats there are part of the result.

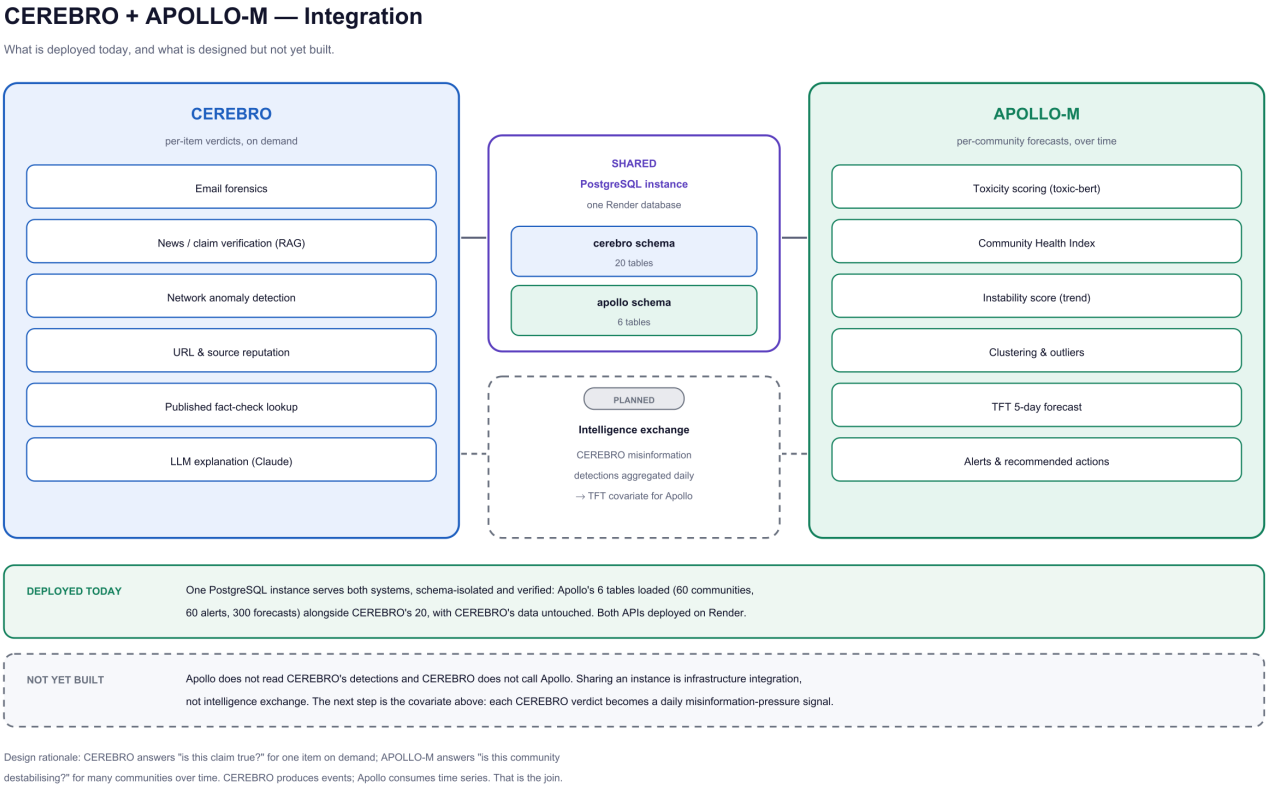
8. Where Everything Lives (current)

Running **locally**, end-to-end, right now:

Component	Location
Frontend dashboard (Streamlit)	`http://localhost:8501`
Backend API (FastAPI) + docs	`http://localhost:8010` (`/docs`)
Database (PostgreSQL - shared with CEREBRO)	`127.0.0.1:5433`, `apollo_db`
Metrics exporter	`http://localhost:9100/metrics`
Prometheus	`http://localhost:9090` (PromQL)
Grafana	`http://localhost:3000` (APOLLO-M dashboard)
Live real-time feed	in-dashboard LIVE LIVE panel (replay)

Planned deployment: backend + database on **Render**; Streamlit dashboard on **Streamlit Cloud** or Render; monitoring local or Grafana Cloud. All connections are environment-variable driven (`DATABASE_URL`, `APOLLO_API_URL`, `CEREBRO_API_URL`), so deployment is a configuration step, not a code change.

9. Integration with CEREBRO



{width=100%}

Figure 3 - What is deployed against what is designed. One PostgreSQL instance serves both systems with `cerebro` and `apollo` schemas side by side; the intelligence exchange between them is specified but not yet built, and is drawn dashed for that reason.

CEREBRO and Apollo-M are **two complementary intelligence systems on one platform**, connected by:

1. **Shared PostgreSQL instance, separate schemas** - CEREBRO's `cerebro` database and Apollo's `apollo_db` live in the same Postgres server (exactly the topology CEREBRO's report recommends). *Implemented and running.*
2. **Shared `model_registry` pattern** - both projects version models the same way.
3. **The same governing principle** - deterministic models decide; the LLM only explains.
4. **A CEREBRO-style misinformation module inside Apollo** - the misinfo classifier feeding the CHI.

Honest boundary: CEREBRO's data domain is email / news / network threats; Apollo-M's is Reddit community health. They therefore share *architecture family* (TFT + RoBERTa + unsupervised), *infrastructure*, and *design principles* - rather than Apollo literally training on CEREBRO's email records. Deployed, Apollo's backend can additionally call CEREBRO's REST API over HTTPS (`CEREBRO_API_URL`) for cross-system enrichment.

---

10. Engineering Record - notable decisions & findings

- **Bugs found by actually running the pipeline (not hidden - fixed).** (1) A `x 100` overflow forced **CHI to 0 for every community**. (2) A dropped **minimum-comments-per-community filter** let sparse subreddits produce coin-flip toxicity. (3) The loader read `nrows=50000` from a CSV **sorted by subreddit name**, so the pipeline only ever analysed communities beginning with "A" - 33 of 831 eligible, selected by alphabet rather than by any criterion. Selection is now the 60 largest communities by volume, a criterion that can be stated and defended. (4) Polarisation and echo-chamber were global constants copied to every community. (5) `compute_real_metrics.py` silently **overwrote** `meso_report.csv` using different CHI weights (70/30/5/5) from the pipeline's (35/30/20/15), so published CHI depended on which script ran last; it now only reads. (6) Three headline metrics were **hardcoded string literals** rather than measurements - see §5.2, §5.3, §5.6.
- **The larger, correct sample disproved the earlier results rather than improving them.** Fixing the alphabetical bias raised per-community volume from ~40 comments to ~4,600 and collapsed the apparent toxicity spread from 0.67-0.83 to 0.543-0.559 - revealing that the original dataset's community differences were noise (§7.1). Reporting that is more useful than the numbers it replaced.
- **RL -> supervised learning** - a documented, defensible substitution (interpretable, no simulation environment required).
- **LLM as a switchable, non-critical layer** - Ollama-first for cost, Claude optional for quality, template fallback so nothing collapses without a key.
- **Real-time processing without a blocked API** - Reddit locked down its public JSON API (403) and gated app creation; a **live replay** streams the real, already-scored corpus through the pipeline in real time, giving genuine real-time processing with no external dependency.
- **Monitoring under a restricted network** - Docker Hub image pulls failed on the build network, so Prometheus + Grafana were installed as **native binaries** (from GitHub / grafana.com), delivering the full monitoring stack without Docker.
- **Shared database under a port conflict** - a native Postgres shadowed port 5432, so the container database was cleanly republished on 5433 (data preserved) to give the host access to the shared instance.

---

11. Current Status - complete vs. remaining

Capability	Status
Multi-layer pipeline (Micro -> Meso -> Unsupervised -> Macro/TFT -> Alerts)	**Runs end-to-end** on the declared simulation (§7.1)
TFT 5-day quantile forecast	**Complete** (bug-fixed, real widening bands); 60 communities
Toxicity metrics (F1 micro/macro) on real labelled data	**Complete, verified** - the one metric on a real corpus
Ground-truth validation harness (`validate_simulation.py`)	**Complete** - detection + forecast measured, `outputs/validation.json`
CHI, instability score, alerts, clusters	**Complete**; CHI's limitation measured and documented (§5.3)
CEREBRO misinfo module inside Apollo	**Not wired** - no corpus, no classifier, not called
GNN (GraphSAGE)	**Not wired** - implemented, never invoked
Moderation recommender (RandomForest)	**Not wired** - actions are a deterministic mapping
Transformer autoencoder anomaly detection	**Not usable** - trained on `torch.randn`, i.e. random noise
Automated test suite	**Absent** - first item of remaining work
Streamlit dashboard + real-time live feed	**Complete & live**
FastAPI backend serving from PostgreSQL	**Complete & live**
CEREBRO Apollo shared-database connection	**Complete & live**
Prometheus + Grafana monitoring	**Complete & live**
LLM explanation (Ollama / Claude switch)	**Complete**
Cloud deployment (Render + Streamlit Cloud)	**Remaining - scheduled** (needs the Claude key for the deployed LLM)
Live Reddit via official API	**Blocked by Reddit policy** - live-replay substitute working
Public monitoring hosting (Grafana Cloud)	Optional - local for the demo
Shared `model_registry` wiring	Optional enhancement
One-command launcher	Convenience - planned

In short: Apollo-M is a working end-to-end predictive pipeline, validated against recorded ground truth on a declared simulation, with its measured limits (§5.3), its unwired modules (above) and its data provenance (§7.1) stated rather than hidden. Several modules named in the architecture are implemented but not yet invoked; the table above says which, and `outputs/metrics.json` marks every unevaluated module rather than assigning it a

number.

---

12. Testing & Verification

What is measured, and on what. Every figure below is written by a script into `outputs/metrics.json` or `outputs/validation.json` and is reproducible by re-running `simulate_data.py` -> the pipeline -> `validate_simulation.py`. Claims that cannot be reproduced that way have been removed from this report rather than restated (see §7.1).

- The toxicity classifier was trained and measured on a **held-out split of the Davidson/Jigsaw corpus** - accuracy 90.1%, F1-micro 0.901, F1-macro 0.702 (4,957 held-out rows). This is the project's one metric on real labelled data.
- **Detection, against planted ground truth** (§7.1): ranking communities by the trend-aware **instability score** gives ROC-AUC **1.000** (top-15 precision and recall 1.00). Ranking by **CHI** gives **0.575**, and by raw toxicity **0.649**. *The 1.000 must not be quoted as accuracy:* the instability score measures a recent-vs-baseline change and the simulation plants a monotonic ramp, so it is looking for the shape it was given. It demonstrates the pipeline is wired correctly and recovers a known signal - nothing more.
- **Forecasting, against planted ground truth:** the TFT - which is never shown the ground-truth file - predicts a rising trend for **15/15** destabilising communities, with slope ROC-AUC **0.732** separating them from the rest, and mean slopes ordered correctly (destabilising +0.0024, stable +0.0009, improving 0.0008). This is the project's strongest independent result.
- The TFT produces genuine **widening p10/p50/p90 bands** (a prior defect that collapsed p10 = p50 = p90 was found and fixed).
- Prometheus is confirmed scraping Apollo metrics via **PromQL**; Grafana renders them in a provisioned dashboard.
- The backend endpoints (`/health`, `/summary`, `/communities`, `/alerts`, `/forecast`) return data from PostgreSQL, verified live.

Not yet verified - stated rather than omitted. There is currently **no automated test suite in `apollo-m`** (an earlier draft of this report claimed standalone tests for the CHI formula, the micro layer and the CEREBRO detector; those tests do not exist and the claim has been withdrawn). Verification today is by reproducible scripts and the ground-truth validation above, not by unit tests. Adding a test suite is the first item of remaining work.

End of report. CEREBRO is the eyes and ears; Apollo-M is the brain. This document records the latter as built - running, measurable, and honest about what remains.